```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Handwritten digit recognition is a computer vision task where a model identifies digits written by humans. It is widely used in applications like bank check processing, postal systems, and OCR (Optical Character Recognition). **In this project**, we build a Convolutional Neural Network (CNN) to recognize digits and deploy it in an interactive interface where users can draw digits and get predictions in real time.

```
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9). Each image is 28×28 pixels. It is a standard benchmark dataset for image classification tasks.

```
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
```

Pixel values are normalized to improve training stability. Images are reshaped to include a channel dimension because CNNs expect 3D input (height, width, channels).

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

**The CNN **extracts features from images using convolution layers. Batch normalization stabilizes training, while pooling reduces image size. Dense layers perform classification. Dropout prevents overfitting.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))
```

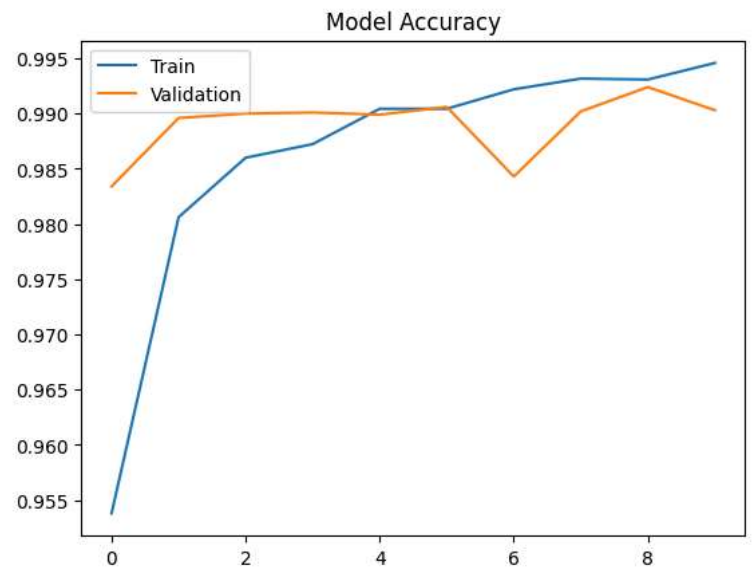
```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 125s 65ms/step - accuracy: 0.9538 - loss: 0.1619 - val_accuracy: 0.9834 - val_loss: 0.0544
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 117s 52ms/step - accuracy: 0.9806 - loss: 0.0686 - val_accuracy: 0.9896 - val_loss: 0.0342
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 143s 52ms/step - accuracy: 0.9860 - loss: 0.0503 - val_accuracy: 0.9900 - val_loss: 0.0406
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 97s 52ms/step - accuracy: 0.9872 - loss: 0.0462 - val_accuracy: 0.9901 - val_loss: 0.0378
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 97s 52ms/step - accuracy: 0.9904 - loss: 0.0328 - val_accuracy: 0.9899 - val_loss: 0.0414
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 159s 60ms/step - accuracy: 0.9904 - loss: 0.0341 - val_accuracy: 0.9906 - val_loss: 0.0381
Epoch 7/10
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 100s 53ms/step - accuracy: 0.9922 - loss: 0.0289 - val_accuracy: 0.9843 - val_loss: 0.0612
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 140s 52ms/step - accuracy: 0.9932 - loss: 0.0251 - val_accuracy: 0.9902 - val_loss: 0.0392
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 141s 52ms/step - accuracy: 0.9931 - loss: 0.0244 - val_accuracy: 0.9924 - val_loss: 0.0379
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 98s 52ms/step - accuracy: 0.9946 - loss: 0.0200 - val_accuracy: 0.9903 - val_loss: 0.0444
```

The model learns patterns from training data. Accuracy measures correct predictions, while loss shows how well the model fits the data.

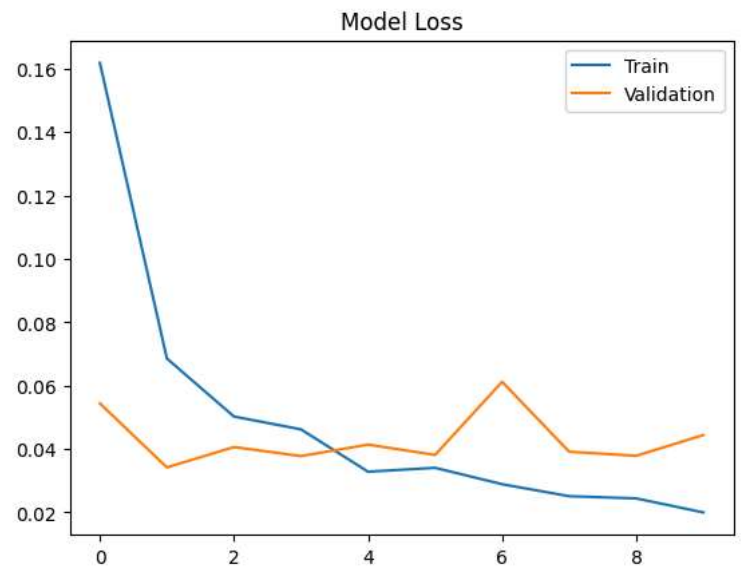
```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.legend(["Train","Validation"])
plt.show()
```



This plot shows how accuracy improves during training. If training accuracy is high but validation is low, it indicates overfitting.

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.legend(["Train","Validation"])
plt.show()
```



Loss represents the model's error. A decreasing loss indicates the model is learning effectively.

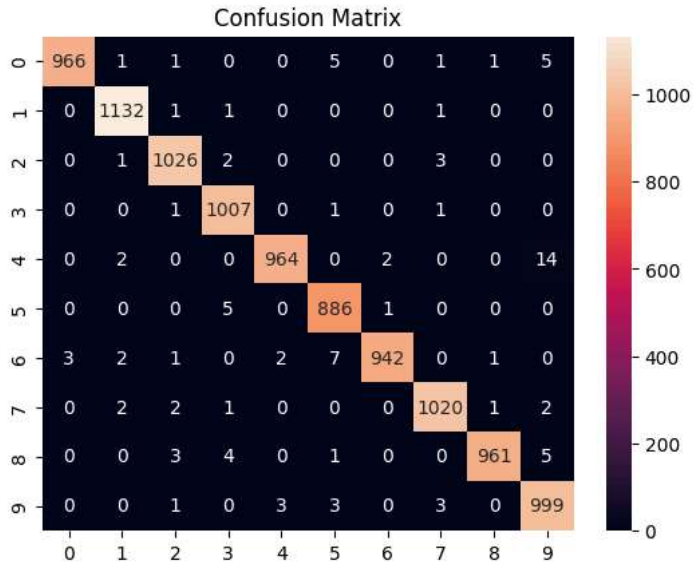
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import numpy as np

pred = model.predict(X_test)
pred_classes = np.argmax(pred, axis=1)

cm = confusion_matrix(y_test, pred_classes)

sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```

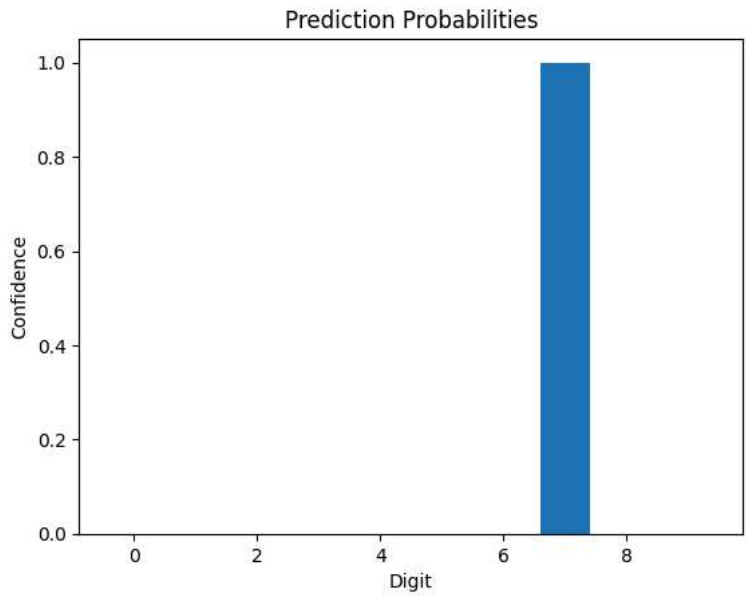
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step
```



The confusion matrix shows how well the model predicts each digit. It helps identify which digits are confused (e.g., 3 vs 5).

```
import matplotlib.pyplot as plt

i = 0
plt.bar(range(10), pred[i])
plt.title("Prediction Probabilities")
plt.xlabel("Digit")
plt.ylabel("Confidence")
plt.show()
```



This chart shows the model's confidence for each digit. It helps understand how certain the model is about its prediction.

```
import joblib
model_path = "/content/drive/MyDrive/Colab Notebooks/CodeAlpha/CodeAlpha_HandwrittenRecognition/task3.h5"
model.save(model_path)
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
```

```
!ls "/content/drive/MyDrive/Colab Notebooks/CodeAlpha/CodeAlpha_HandwrittenRecognition"
```

```
handwritten_model.h5  Handwritten_Recognition.ipynb  task3.h5  task3.ipynb
```